

Question 1

Answer

A **clear prompt** provides the LLM with a well-defined role, specific instructions, decision rules, and the expected output format. An **unclear prompt** is vague, lacks constraints, and leaves the model to make assumptions, which often results in inconsistent or incorrect responses.

In the food delivery chatbot scenario, the unclear prompt does not specify when a refund should be approved or denied. As a result, the LLM may approve refunds without checking order status or reject valid requests inconsistently.

Change 1: Define Clear Decision Rules

Instead of using a vague prompt like:

"Help the customer with their refund request."

use a structured prompt such as:

You are a Food Delivery Customer Support Agent.

Refunds should only be approved if:

- The order contains missing items.
- The wrong item was delivered.
- The order was cancelled before food preparation.
- The restaurant cancelled the order.

Otherwise, politely deny the refund and explain the reason.

Why this matters:

Clearly defining business rules ensures the LLM follows the company's refund policy instead of making inconsistent decisions. It reduces ambiguity and improves response consistency.

Change 2: Specify the Response Format

Instead of allowing free-form responses, define a structured output such as:

Decision: Approved / Rejected

Reason:

<Explanation>

Next Step:

<What the customer should do next>

Why this matters:

A fixed response format produces consistent outputs that are easier to read, validate, and integrate into applications. It also prevents the model from generating unnecessary or irrelevant information.

Conclusion

A clear prompt includes:

- A defined role.
- Explicit decision rules.
- Output formatting instructions.
- Business constraints.

These improvements reduce ambiguity, improve consistency, and produce more reliable responses from the LLM.

Question 2

Answer

Zero-shot prompting asks the LLM to complete a task without providing any examples. The model relies entirely on its pre-trained knowledge to classify customer complaints.

Example:

Classify the complaint into one of the following categories:

- Late Delivery
- Wrong Item
- Missing Item
- Poor Quality

Complaint:

"My order arrived after two hours."

Output:

Few-shot prompting provides several labelled examples before presenting the new complaint. These examples demonstrate the expected input-output pattern and guide the model toward more accurate classifications.

Example:

Input: My order arrived 45 minutes late.

Output: Late Delivery

Input: I received a burger instead of pizza.

Output: Wrong Item

Input: My drink was missing.

Output: Missing Item

Complaint:

"My dessert was not included."

Output:

Comparison

Zero-Shot Prompting	Few-Shot Prompting
No examples are provided.	Includes labelled examples.
Simpler and faster.	Provides better guidance to the model.
May produce inconsistent predictions.	Usually produces more accurate and consistent results.
Uses very little context.	Uses part of the context window for examples.

Which Approach Would I Choose?

Given that there are **200 labelled examples** but the LLM has a **limited context window**, I would choose **few-shot prompting**.

Instead of including all 200 examples, I would select a **small set of representative examples** that covers all four complaint categories. This provides enough guidance while staying within the model's context limit. Few-shot prompting generally improves classification accuracy because the model learns the desired input-output pattern from the examples.

Factors That Determine the Number of Few-Shot Examples

The number of examples included depends on several factors:

1. **Context Window Size**
The prompt, examples, and user complaint must all fit within the model's maximum token limit.
2. **Task Complexity**
More complex classification tasks may require additional examples to illustrate different situations.
3. **Category Coverage**
Each complaint category should be represented by at least one high-quality example to help the model distinguish between them.
4. **Example Quality**
Well-written and diverse examples are more valuable than simply including a large number of similar examples.

Question 3

Answer

Chain-of-Thought (CoT) prompting is a prompt engineering technique that encourages the LLM to solve a problem by reasoning through it step by step before producing the final answer. Instead of giving an immediate response, the model evaluates each factor in a logical sequence.

In the delivery route optimization scenario, the assistant must consider multiple variables such as delivery addresses, traffic conditions, and priority levels. A direct prompt may cause the model to overlook some of these factors, resulting in incorrect or inconsistent routes. Using CoT prompting helps the model analyze each variable systematically, leading to more accurate and reliable route recommendations.

Example Prompt

Instead of using:

Find the best delivery route for these five orders.

Use a Chain-of-Thought prompt:

You are a delivery route optimization assistant.

Determine the optimal delivery sequence by following these steps:

1. Review the priority level of each order.
2. Compare estimated traffic delays for each location.
3. Group nearby delivery addresses where possible.
4. Choose the sequence that minimizes travel time while delivering high-priority orders first.
5. Explain your reasoning for each decision.

6. Finally, provide the recommended delivery order.

This prompt encourages the model to consider all relevant factors before arriving at the final route.

Limitation

One limitation of Chain-of-Thought prompting is that it increases the number of tokens in both the prompt and the response. This results in higher computational cost, longer response times, and greater use of the model's context window, which can be a limitation for large or complex tasks.

Question 4

Answer

For this use case, I would choose **Retrieval-Augmented Generation (RAG)** instead of fine-tuning the LLM.

The policy handbook is approximately **150 pages** and is **updated every quarter**. RAG is more suitable because it retrieves the latest information directly from the document without modifying the model itself.

Reason 1: Handles Frequent Document Updates

With RAG, whenever the handbook is updated, only the document embeddings and vector database need to be regenerated. The LLM continues to use the most recent version of the handbook without requiring retraining.

In contrast, fine-tuning would require collecting the updated data, retraining the model, validating it, and deploying the new version after every handbook update. This process is time-consuming and expensive.

Reason 2: Provides Accurate and Current Information

RAG retrieves only the sections of the handbook that are relevant to the user's query. The generated answer is therefore based on the latest official document, reducing the risk of outdated or hallucinated information.

A fine-tuned model stores knowledge in its parameters. If the handbook changes after training, the model may continue producing outdated answers until it is fine-tuned again.

When Fine-Tuning Would Be Better

Fine-tuning would be a better choice if the goal were to teach the model a **specialized behavior or writing style** rather than retrieve frequently changing information.

For example, if the company wanted the AI assistant to always respond in a specific customer service tone or perform a domain-specific classification task using thousands of labeled examples, fine-tuning would be more appropriate because the desired behavior becomes part of the model itself.

Conclusion

For a frequently updated policy handbook, **RAG** is the better solution because it is easier to maintain, provides up-to-date information, and avoids the repeated cost of retraining. Fine-tuning is more suitable when the objective is to permanently teach the model a new skill or behavior.

Question 5

Answer

In a **Retrieval-Augmented Generation (RAG)** pipeline, documents are divided into smaller pieces called **chunks** before they are converted into embeddings and stored in a vector database.

Chunk size determines how much text is included in each chunk, while **chunk overlap** determines how many words or sentences are shared between consecutive chunks.

In this scenario, important information is being split across different chunks. For example, the dish name may appear in one chunk while its price appears in the next, making it difficult for the retrieval system to return complete information.

Improving Retrieval Quality

To improve retrieval quality, I would:

1. **Increase the chunk size**

Larger chunks allow related information, such as a dish description and its price, to remain together in the same chunk. This gives the retrieval system more complete context.

2. **Increase the chunk overlap**

Increasing the overlap ensures that important information appearing near chunk boundaries is included in both neighboring chunks. This reduces the chance of splitting related content across separate chunks.

For example:

- Chunk Size: **100 words** → **200 words**
- Chunk Overlap: **20 words** → **50 words**

This configuration increases the likelihood that both the dish name and its price appear together during retrieval.

Trade-Off

The main trade-off is increased storage and computational cost.

Larger chunks and greater overlap produce more text per embedding and create duplicate content across chunks. This increases the number of embeddings stored in the vector database, consumes more memory, and may slightly slow retrieval and indexing. However, the improvement in retrieval accuracy and context quality often outweighs these additional costs.

Question 6

Answer

Both **FAISS** and **ChromaDB** are vector stores used to perform similarity search on embeddings, but they differ in their features and use cases.

FAISS

FAISS (Facebook AI Similarity Search) is an open-source library developed by Meta for efficient similarity search over large collections of vectors. It is designed for **high-speed nearest-neighbor search** and is commonly used in RAG applications where fast retrieval is the primary requirement. However, FAISS mainly focuses on vector indexing and does not provide built-in document management or metadata handling.

Best scenario for FAISS:

- Large-scale semantic search where maximum retrieval speed is required.
- Applications with mostly static data that does not change frequently.

ChromaDB

ChromaDB is a vector database designed specifically for LLM and RAG applications. In addition to storing embeddings, it stores **documents, metadata, and unique IDs**, making it easier to add, update, delete, and filter records. It provides built-in persistence and document management features.

Best scenario for ChromaDB:

- RAG applications where documents are frequently added, updated, or deleted.
 - Systems that require metadata filtering (for example, retrieving complaints only from the last 30 days or only unresolved complaints).
-

Would Switching to ChromaDB Solve the Outdated-Record Problem?

Switching to ChromaDB **alone would not completely solve the problem.**

The issue is that **resolved complaints remain in the vector store**, so they continue to appear during retrieval. While ChromaDB makes it easier to update or delete records using document IDs and metadata, outdated records will still be retrieved if they are never removed or updated.

Therefore, simply changing the vector store does not automatically eliminate outdated information.

Additional Step Required

Regardless of whether FAISS or ChromaDB is used, the system must implement a **data synchronization or index maintenance process**.

This process should:

- Remove embeddings for resolved complaints when they are no longer needed.
- Update embeddings whenever complaint information changes.
- Rebuild or refresh the vector index periodically if necessary.

Keeping the vector store synchronized with the latest complaint database ensures that retrieval results remain accurate and up to date.